

Manual for Stridulant: a Python Toolkit for Bioacoustic Analysis of Non-Vocal Organisms

Contents

1	Installation	1
1.1	Download	1
1.2	Install	1
1.3	Updating the program	2
2	Basic functionality	3
2.1	Snippets	4
2.2	Spectrograms	4
2.3	Normalization	7
2.4	Filters	7
2.5	Finding events and extracting features	7
3	Feature-based scans	9
3.0.1	Included functions	10
3.0.2	User-defined feature-based scans	11
4	Batch processing	12
5	Convolutional neural network	12

1 Installation

1.1 Download

Please download and unpack the anonymized version of *Stridulant* from Zenodo in your folder of choice.

1.2 Install

The easiest way is to install Stridulant with pip install. With a python interpreter (Bash, Spyder, Studio Visual Code or whichever you are using) navigate to the Stridulant folder.

```
# Navigate to the Stridulant folder, for example: #  
cd stridulant  
  
# Install package #  
pip install -e .
```

```
# To uninstall, run #  
pip uninstall -y stridulant
```

Now that the package is installed you can import it like any other module in Python.

```
import stridulant as st
```

Stridulant is now ready to use. Note that many functions in this manual have the same arguments. Their definitions are consistent throughout the manual and are thus not repeated in each section.

1.3 Updating the program

Anytime an update is available, go to git bash and execute:

```
# Navigate to the Stridulant folder #  
cd path_to_folder/stridulant  
  
# Update #  
git pull origin master
```

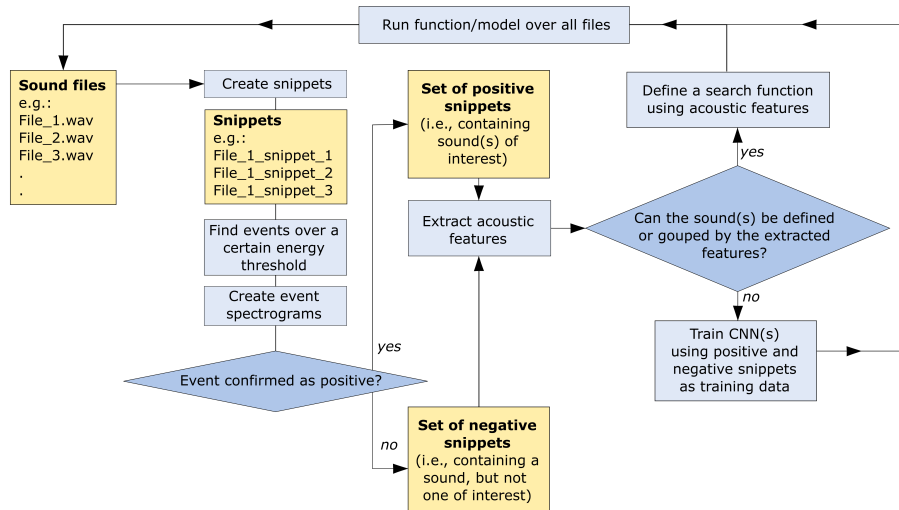


Figure 1: Basic workflow of Stridulant using the snippet logic. The whole loop may be repeated several times until the snippet sets are large enough to produce reliable functions or models. Our package provides a wrapper for a basic convolutional neural network (CNN) for which we also provide example scripts.

2 Basic functionality

All code in this manual can be found in the Stridulant installation folder under Tutorials

Stridulant’s main task is to ease the preparation of sounds for custom feature-based scans or for AI training (Fig. 1). To do this, audio files are broken down into snippets. These snippets are fragments of a sound file that can then be easily handled and saved.

2.1 Snippets

Single snippets are created as follows:

```
# Load an audio file #
audio, sr = st.load_audio(path_to_audio)

# Create a snippet #
Snippet = st.create_snippet(audio, sr, start_time, duration)

# Play snippet #
Snippet.play()

# Save snippet #
Snippet.save(source_name, output_dir, verbose, metadata)
```

path_to_audio	Path of audio file. Supported files are WAV and FLAC.
source_name	Base name for the saved file (typically the source file name).
output_dir	Directory where the snippet will be saved.
start_time	Start time of the snippet within the audio file in seconds.
duration	Length of snippet in seconds.
metadata	If True, appends snippets name with the snippet start time. Default is True
verbose	If True, prints a message when the snippet is saved. Default is True

2.2 Spectrograms

Snippets can be used to create spectrograms as follows:

```
# Create spectrogram object #
spectrogram = Snippet.spectrogram (spec_type, n_fft, n_mels, hop_length, window,
    env_smooth)

# Plot spectrogram #
spectrogram.plot(color)
spectrogram.plot_events(events)

# Save spectrogram object #
spectrogram.save_img(name, output_dir, with_labels, events)
```

spec_type	Type of the spectrogram ('mel', 'fft' or 'hilbert'). Default is 'mel' (Fig. 2)
hop_length	Overlap between windows. Bigger numbers (up to the window size) will make the process faster at the expense of temporal resolution, but also will make windows more independent from each other. Big numbers are best for power analysis, smaller numbers are better for very short sounds and high temporal resolution (visualization and perhaps CNNs)
window	Shape of the window. Windows are related to leakage, which is a 'spill' of energy from a frequency bin to the adjacent one. The default is boxcar, which has a lot of leakage, but also the best frequency resolution. Check Librosa documentation for more options.
n_mels	Use when spec_type is set to 'mel'. Higher numbers increase the resolution in the frequency domain. It is best to use powers of 2
n_fft	Use when spec_type is set to 'fft'. This is the size of the window analyzed. Bigger numbers will increase resolution in frequency and smaller numbers increase resolution in time. It is a lot more efficient if the number is a power of 2.
env_smooth	Smooth factor for the Hilbert envelope. It is a number in milliseconds that smooths the peaks on that time range. Make sure that $\text{int}(\text{env_smooth}/1000*\text{sr}) > 0$. Check the sampling rate (sr) of the loaded audio file with <i>print(sr)</i> , or of the snippet with <i>print(snippet.sr)</i> . Often numbers around 10 or so give a good trade-off between the smoothness of the line and the retention of features. Smaller numbers will give higher details, bigger numbers smoother lines. Target sounds in the ultrasonic range should employ even smaller numbers, for example 1 or even 0.1.
color	Colormap for the spectrogram. Use <i>plt.colormaps()</i> to learn about your options.
with_labels	Add axis labels and legend to the spectrogram. Default is True.
events	List of events created by <i>find_events()</i> (see Section 2.5). If provided, shows where in the snippet acoustic events are found. Only applicable to Hilbert spectrograms (Fig. 3).

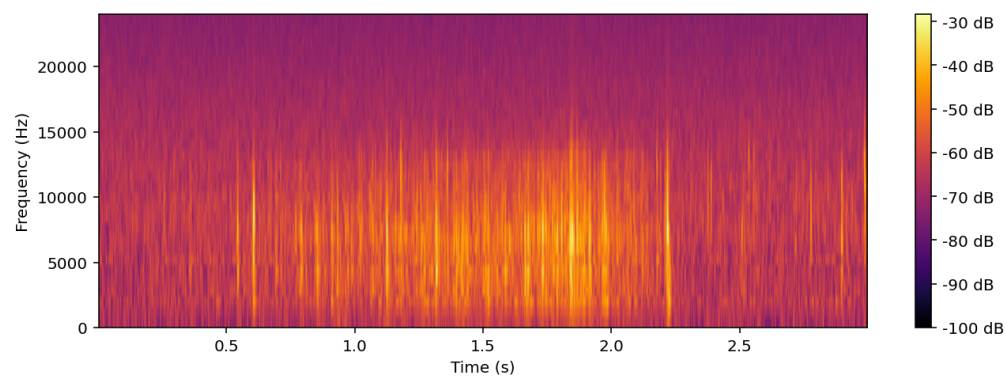


Figure 2: Mel spectrogram.

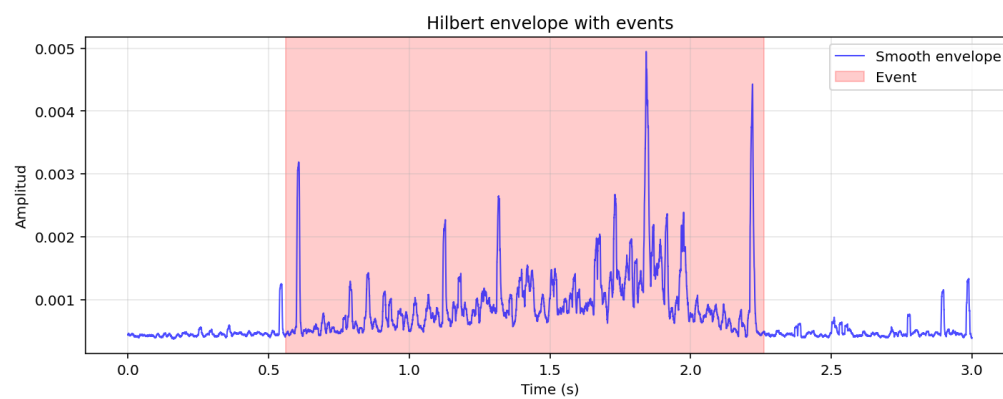


Figure 3: Hilbert spectrogram with event detection.

2.3 Normalization

There are three normalization functions that operate at different levels, i.e. they pick the maximum to normalize to from a different set: 1) within one snippet, 2) within one audio file and 3) across several audio files based on the global maximum. All levels of normalization are also available in the Batch Processing function (see section 4).

```
# Normalize within a single snippet #
Snippet_1.normalize = Snippet_1.normalize()

# Normalize within one audio file #
audio_normalize = st.normalize_audio(audio, target_max)

# Normalize across files #
normalize_global(input_dir, output_dir, target_max)
```

output_dir Where you want the normalized files to be saved.
audio The input audio signal.
target_max max percentage for the peak value (default 75%)

2.4 Filters

High and lowpass filters can be added, for example to filter out rain or to focus on the ultrasonic range.

```
Snippet_filtered = Snippet

# Highpass filter #
Snippet_filtered.audio = st.highpass_filter(Snippet.audio, sr, cutoff, order)

# Lowpass filter #
Snippet_filtered.audio = st.lowpass_filter(Snippet.audio, sr, cutoff, order)
```

audio Input audio signal.
sr Sampling rate of the audio
cutoff Cutoff frequency in Hz.
order Filter order. Default is 8 (48 dB/octave roll-off).

2.5 Finding events and extracting features

The following functions allow the user to find events above an energy percentile threshold and extract acoustic features. All the functions in this section work best on short audio files (i.e. up to around 10 seconds).

`Batch_feature_finder()` allows the user to extract all features from a folder of snippets instead of single audio files. This is useful after a set of positive and negative snippets has been made manually in order to inspect what differentiates target and non-target sounds. See the tutorial *Prepare snippet sets*.

Further, the batch processing function (Section 4) can be used to scan whole folders of audio for events above an user-defined energy threshold.

```

# Find events within a snippet #
events = Snippet.find_events(min_event_duration, threshold_percentile, env_smooth)
Snippet.extract_features(events[0], pulse_dist, env_smooth)

# Export events and features to CSV #
feature_finder(output_folder, output_name, audio, min_event_duration,
               threshold_percentile, env_smooth, highpass_filter_value, lowpass_filter_value,
               highpass_order, lowpass_order)

# Batch extracting acoustic features from a folder of snippets #
batch_feature_finder(output_folder, snippet_folder, output_name, df_format,
                    min_event_duration, threshold_percentile, env_smooth, highpass_filter_value,
                    lowpass_filter_value, highpass_order, lowpass_order)

```

pulse_dist	Minimum time between detectable pulses in milliseconds. Must be > 0. Controls pulse detection sensitivity.
audio	Either a snippet object or a path to an audio file. Not recommended on audio files longer than approximately 10 seconds.
min_event_duration	Minimum duration of events in seconds.
threshold_percentile	Percentile value for energy threshold (0-100).
snippet_folder	Path to snippet folder.
df_format	Create a 'long' or 'wide' (default) data frame.

The extracted features and their meaning can be found in Table 1.

Table 1: Sound features included in *Stridulant* that can be used to describe sounds and define feature-based scans. ‘Event’ refers to a single acoustic event, which within the *Stridulant* pipeline is defined as an audio signal where the Hilbert energy envelope exceeds a user-defined percentile threshold. Because *Stridulant* is targeted mainly towards non-vocalization sounds, we incorporated several *pulse* descriptors (features six to ten). Pulses are short bursts of energy that are common in insect sounds. These descriptors quantify properties of the pulses within an event, such as their number, duration, spacing, and temporal distribution.

	Feature name	Description	Unit
1	event duration	Duration of the event	Seconds
2	event energy	Peak amplitude of the smoothed envelope	Typically unitless, unless audio is calibrated
3	attack rate	Rate at which amplitude rises in the first 20% of the event envelope	Amplitude units per sample
4	duty cycle	Fraction of event duration above an envelope threshold	Fraction (0–1)
5	dynamic range	Difference between the loudest and quietest spectral amplitudes	dB
6	avg pulse duration	Mean pulse duration	seconds
7	avg pulse interval	Mean time between pulses	seconds
8	pulse count	Number of pulses	–
9	pulse density	Pulses per second	–
10	pulse regularity	Rhythm consistency	Standard deviation of intervals between pulses
11	frequency range	Bandwidth containing 90% of spectral energy	Hz
12	spectral centroid mean	Dominant frequency	Hz
13	spectral roll off	Frequency at which 85% of energy falls below	Hz
14	tonal variation	Frequency stability	Standard deviation of the mean of centroid

3 Feature-based scans

Based on the acoustic features of a sound, feature-based scans can be used to determine whether it fits the target sound or not. We include two predefined functions (for ant stridulation and plant UAEs) and work through one example

of how to make your own using worm sounds. See the tutorial *feature-based scans*.

3.0.1 Included functions

The included feature-based scan provide a starting off point, but they will likely have to be fine-tuned to the users specific audio files. The function *is_stridulation* allows for event coupling, meaning that two weak events will be combined together, if they are not separated more than *coupled_gap*. The standard *min_event_duration* still has to be defined in order to do the initial event scan.

```
# Ant stridulation #
is_stridulation(snippet, min_event_duration, threshold_percentile, min_pulses,
                min_regularity, uncoupled_min_duration, max_duration, sustain, pulse_dist,
                sp_range, env_smooth, enable_coupled, coupled_min_duration, coupled_gap)
# Returns True if a stridulation may be present in the snippet.

# Tree cavitation #
is_cavitation(snippet, spectral_rolloff_min, min_energy, min_event_duration,
              max_duration, max_energy, pulse_dist, env_smooth, threshold_percentile)
# Returns True if a cavitation—like sound may be present in the snippet.
```

snippet	Snippet object
min_pulses	Minimum number of individual pulses required within the event. Default is 6
min_duration	Minimum event duration in seconds for strong individual events. Use 0 for no minimum. Default is 0.5.
max_duration	Maximum event duration in seconds. Use None for no maximum. Default is 1
pulse_dist	Minimum distance between pulses in milliseconds. Default is 20.
sp_range	Valid frequency range for spectral centroid in Hz (min, max).
enable_coupled	Whether to enable detection of coupled weak events. Default is True.
coupled_min_duration	Minimum duration for events in coupled detection. Allows shorter events to be considered only when looking for coupled pairs. Default is 0.22.
coupled_gap	Maximum time gap between consecutive weak events in seconds. Default is 0.6.
spectral_rolloff_min	Frequency at which at which 85% percent of energy lays below. I.e., a minimum set at 23000 ensures that at least 15% of the events' energy is in the ultrasonic range. Default is 23000.

3.0.2 User-defined feature-based scans

Below is a most basic example of a user-defined feature-based scan. For a more complicated example, see the *feature-based scans* tutorial. Ensure that throughout the process of developing a batch processing settings, the same values for `env_smooth`, `threshold_percentile` and `pulse_dist` values are used, as differences in these parameters affect the outcomes.

```
# Define a function called is_test()
def is_test(snippet,
            env_smooth = 10,
            threshold_percentile = 20,
            min_energy = 1,
            pulse_dist = 20,
            min_event_duration = .5,
            ...
            # Add more arguments here
            ):

    # Detect events
    events = snippet.find_events(
        min_event_duration,
        threshold_percentile,
        env_smooth)

    if not events:
        return False

    # Extract features from events
    else:
        for event in events:
            features = snippet.extract_features(event, pulse_dist = pulse_dist)

            # Return True if any of the events match the arguments
            if (features["event_energy"] > min_energy):
                return True

        return False

# Apply function to a snippet
snippet.is_test()
```

4 Batch processing

This function allows users to scan whole folders for target sounds. It is designed to be able to handle interruptions. It does this by reading and writing a log file, which is also where the results are saved (Table 2). You can also use this function to save all events above an energy threshold, i.e., without a method screening for target sounds.

```
# Batch scanning for target sounds #
batch(method, output_folder, log_name, snippet_duration, folder_list, normalize,
      target_max, highpass_filter_value, lowpass_filter_value, highpass_order,
      lowpass_order, min_event_duration, threshold_percentile, env_smooth,
      make_spectrograms, spec_type, n_fft, n_mels, hop_length, with_labels,
      make_snippets, metadata, log_save)
```

method	Either a preloaded function ('is_cavitation' or 'is_stridulation') or a user defined function (i.e., is_worm_rumble, see the tutorial <i>Batch processing</i>). Can also be set to None to save all events without running a feature-based scan.
output_folder	Directory where the log, snippets and spectrograms will be saved.
log_name	Name for the log. Default is 'snippet.log'.
folder_list	List of folders to scan for target sounds. Works recursively through all subfolder.
normalize	Type of normalization to be done. Options are 'snippet', 'file' or 'global'. See section 2.3 for details. Default is no normalization.
make_spectrograms	If True, saves all spectrograms in folders, sorted by whether they contain the target sound or not. Default is False.
make_snippets	Saves snippets of events. Options are 'all' for both negative and positive snippets, 'true_only' and 'false_only' for only saving positive or negative snippets, and 'random' for a random 1% of snippets. Default is False.
log_save	Specifies which results are saved in the log. Options are 'all' (default), 'true_only', 'false_only'.

5 Convolutional neural network

If scanning audio files for target sounds using signal processing is not appropriate for the data in question, you may try the basic CNN model that is included in this package (see the tutorial *CNN training and use*).

The model included in this package is designed for binary classification. That means that for each spectrogram, it will decide either True when a target sound is present and False if it is not.

Table 2: Example of the log file created by `batch()`. `Snippet_start_time` indicates the start time within the original audio file. In this case, snippet duration was set to 2 seconds. Event is Yes when at least one event is present in the snippet. Results is TRUE if at least one of the events in the snippet matches the target sound.

Original_audio_path	Snippet_start_time	Snippet_end_time	Event	Result
Ant_stridulation.wav	0	2	Yes	FALSE
Ant_stridulation.wav	2	4	Yes	TRUE
Ant_stridulation.wav	4	6	Yes	TRUE
Ant_stridulation.wav	6	8	No	NA
Ant_stridulation.wav	8	10	Yes	FALSE

```
from stridulant import train_model as tm, classify_spectrograms as cs
```

```
## Train model
```

```
mod, hist = tm.train_model(train_dir, batch_size, epochs, learning_rate,
                           class_weights)
```

```
## Run model
```

```
cs.classify_spectrograms(spectrogram_folder, model)
```

train_dir	Path to the directory containing the training images. Should contain one folder with positive and one with negative spectrograms.
target_size	The target size to which each input image is resized (default is (128, 128)).
batch_size	The number of images per batch used during training (default is 32).
epochs	The number of epochs to train the model (default is 5).
learning_rate	The learning rate for the optimizer (default is 0.001).
class_weights	A dictionary of class weights for handling class imbalance. If None, the class weights are computed based on the class distribution.